

程序设计实习

第十四讲

标准模板库2

杨 帅

williamyang@pku.edu.cn

■ 大作业

- 04月25日 - 将1页项目功能设计文档提交相应负责助教，以确认功能设计合理性
- 稍后会公布对应的助教

■ 期中考试

- 04月23日：标准模板库2
- 04月30日：C++11和高阶应用
- 05月02日：五一放假
- 05月07日：五一放假
- 05月14日：期中考试（暂定）

■ 上机安排

- 05月03日：五一放假
- 05月10日：模拟考试（暂定）

■ 对于有 n 个元素的以下容器，大多数情况下，哪种操作速度最快？

- A 在deque头部添加一个元素
- B 在set中查找元素
- C 在vector头部添加一个元素
- D 在map中删除元素

■ 对于有 n 个元素的以下容器，大多数情况下，哪种操作速度最快？

- A 在deque头部添加一个元素
- B 在set中查找元素
- C 在vector头部添加一个元素
- D 在map中删除元素

■ 下列类模板不支持迭代器的是

A list

B stack

C deque

D set

■ 下列类模板不支持迭代器的是

A list

B stack

C deque

D set

- 以下说法哪个是正确的？
- A 双向迭代器可以和整数相加
 - B 两个双向迭代器可以相减
 - C 两个双向迭代器可以比较大小
 - D `list`上面的迭代器不能比较大小

■ 以下说法哪个是正确的？

- A 双向迭代器可以和整数相加
- B 两个双向迭代器可以相减
- C 两个双向迭代器可以比较大小
- D **list**上面的迭代器不能比较大小

■ 关于顺序容器迭代器的操作，不正确的是

A `vector<int>::iterator iter1, iter2; iter1 < iter2;`

B `list<int>::iterator iter1, iter2; iter1 < iter2;`

C `vector<int>::iterator iter1, iter2; iter1 -= 3;`

D `deque<int>::iterator iter1; iter1 += 5;`

■ 关于顺序容器迭代器的操作，不正确的是

A `vector<int>::iterator iter1, iter2; iter1 < iter2;`

B `list<int>::iterator iter1, iter2; iter1 < iter2;`

C `vector<int>::iterator iter1, iter2; iter1 -= 3;`

D `deque<int>::iterator iter1; iter1 += 5;`

- 关联容器：set和multiset
- 关联容器：map和multimap
- 容器适配器
 - stack / queue / priority_queue
- STL中的算法

- 关联容器： `set`和`multiset`
- 关联容器： `map`和`multimap`
- 容器适配器
 - `stack` / `queue` / `priority_queue`
- STL中的算法

■ `set` / `multiset` / `map` / `multimap`

- 内部元素**有序排列**，新元素插入的位置取决于它的值
- 查找速度快

■ 除了各容器都有的函数外，还支持以下成员函数：

- `find`：查找等于某个值的元素 （ $x < y$ 和 $y < x$ 同时不成立即为相同）
- `lower_bound`：查找下界
- `upper_bound`：查找上界
- `count`：计算等于某个值的元素个数
- `insert`：插入元素或者区间

■ 定义

```
template<class Key, class Pred = less<Key>, class A = allocator<Key>>  
class multiset { ... };
```

- 第一个参数 **Key**: 容器中每个元素类型
- 第二个参数 **Pred** 是个 函数对象
 - Pred决定了multiset中的元素, "一个比另一个小"是怎么定义的
即 $\text{Pred}(x, y)$ 如果返回值为true, 则x比y小
 - Pred的缺省类型是 **less<Key>**

■ 定义

```
template<class Key, class Pred = less<Key>, class A = allocator<Key>>
class multiset { ... };
```

■ less模板的定义:

```
template<class T>
struct less : public binary_function<T, T, bool> {
    bool operator()(const T& x, const T& y)
    { return x < y ; } const;
}; //less模板是靠 < 来比较大小的
```

```
class A{  
    ...  
};  
multiset <A> a;
```

就等效于

```
multiset<A, less<A>> a;
```

- 由于 **less**模板是用 **<** 进行比较的，
所以这要求 **A** 的对象能用 **<** 比较，即必须重载 **<**

multiset的用法

《程序设计实习》

//出错的例子:

```
#include <set>
using namespace std;
class A { };
main() {
    multiset<A> a;
    a.insert(A()); //error
}
```

//编译出错是因为，插入元素时，multiset会将被插入元素和已有元素进行比较，以决定新元素的存放位置

//本例中缺省就是用less<A>函数对象进行比较，然而less<A>函数对象进行比较时，**前提是A对象能用 < 进行比较**

//但本例中没有重载 <

multiset的成员函数

《程序设计实习》

```
iterator find(const T & val);
```

// 在容器中查找值为val的元素，返回其迭代器；如果找不到，返回end()

```
iterator insert(const T & val);
```

// 将val插入到容器中并返回其迭代器

```
void insert(iterator first, iterator last);
```

// 将区间[first, last)插入容器

```
int count(const T & val);
```

// 统计有多少个元素的值和val相等

```
iterator lower_bound(const T & val);
```

// 查找一个最大的位置 it，使得[begin(), it) 中所有的元素都比 val 小

```
iterator upper_bound(const T & val);
```

// 查找一个最小的位置 it，使得[it, end()) 中所有的元素都比 val 大

```
pair<iterator, iterator> equal_range(const T & val);
```

// 同时求得lower_bound和upper_bound

■ 预备知识: **pair**模板 - 用于生成 key-value对

```
template<class _T1, class _T2>
struct pair{
    _T1 first;
    _T2 second;
    pair(): first(), second() { }    //无参数构造函数初始化
    pair(const _T1& __a, const _T2& __b): first(__a), second(__b) { }
    template<class _U1, class _U2>
    pair(const pair<_U1, _U2>& __p): first(__p.first), second(__p.second){ }
};
```

第三个构造函数: `pair<int, int> p(pair<double, double>(5.5, 4.6));`

`// p.first = 5, p.second = 4`

■ pair模板类支持如下操作：

■ `pair<T1, T2> p1;` 创建一个空的pair对象

→ 它的两个元素分别是T1和T2类型，采用值初始化

■ `pair<T1, T2> p1(v1, v2);` 创建一个pair对象

→ 它的两个元素分别是T1和T2类型

→ 其中first成员初始化为v1，second成员初始化为v2

■ `make_pair(v1, v2);` 以v1和v2值创建一个新的pair对象

→ 其元素类型分别是v1和v2的类型

```
pair<int, string> p4 = make_pair(200, "Hello");  
cout << p4.first << ", " << p4.second << endl; // 200, Hello
```

multiset的用法

《程序设计实习》

- 从 `begin()` 到 `end()` 遍历一个 `multiset` 对象，
- 就是从小到大遍历各个元素

multiset的用法

```
#include <set> //使用multiset需包含此文件
#include <iostream>
#include <iterator>
using namespace std;
class MyLess;
class A {
private:
    int n;
public:
    A(int n_ ) { n = n_; }
    friend bool operator< ( const A & a1, const A & a2 )
    { return a1.n < a2.n; }
    friend ostream & operator<< ( ostream & o, const A & a2 )
    { o << a2.n; return o; }
    friend class MyLess;
};
class MyLess {
public:
    bool operator()( const A & a1, const A & a2) {
        return ( a1.n % 10 ) < ( a2.n % 10 );
    }
};
```


multiset的用法

《程序设计实习》

```
typedef multiset<A> MSET1;
typedef multiset<A, MyLess> MSET2;
// MSET2元素的排序规则与MSET1不同

int main() {
    const int SIZE = 5;
    A a[SIZE] = { 4, 22, 19, 8, 33 };
    MSET1 m1;
    m1.insert(a, a+SIZE); //注意set添加元素的函数与vector不同
    m1.insert(22);         //vector要指定插入起始位置
    cout << "1) " << m1.count(22) << endl;
    MSET1::const_iterator p;
    cout << "2) ";
    for (p = m1.begin(); p != m1.end(); p++)
        cout << *p << ", ";
    cout << endl;
}
```

- 1) 2
- 2) 4, 8, 19, 22, 22, 33,

multiset的用法

《程序设计实习》

```
ostream_iterator<A> output(cout, ", ");
MSET2 m2;
m2.insert(a, a+SIZE);
cout << "3) " ;
copy(m2.begin(), m2.end(), output); //COPY函数
cout << endl;
MSET1::iterator pp = m1.find(19);
if( pp != m1.end() ) //找到
    cout << "found" << endl;
cout << "4) " ;
copy(m1.begin(), m1.end(), output);
pair<MSET1::iterator, MSET1::iterator> pr;
cout << endl;
cout << "5) ";
cout << * m1.lower_bound(22) << ", ";
cout << * m1.upper_bound(22) << endl;
pr = m1.equal_range(22);
cout << "6) " << * pr.first << ", " << * pr.second;
return 0;
```

```
}
```

3) 22, 33, 4, 8, 19,
found
4) 4, 8, 19, 22, 22, 33,
5) 22, 33
6) 22, 33

■ 定义

```
template<class Key, class Pred = less<Key>, class A = allocator<Key>>
class set { ... }
```

- 插入set中已有的元素时，插入不成功
- 与multiset的区别：是否允许重复元素
- 与map的区别：是否显式定义key
 - set/multiset 使用元素本身作为key

set的用法示例

```
#include <set>
#include <iostream>
#include <iterator>
using namespace std;
int main() {
    typedef set<double, less<double>> double_set;
    const int SIZE = 5;
    double a[SIZE] = {2.1, 4.2, 9.5, 2.1, 3.7 };
    double_set doubleSet(a, a+SIZE);
    ostream_iterator<double> output(cout, " ");
    cout << "1) ";
    copy(doubleSet.begin(), doubleSet.end(), output);
    cout << endl;
    pair<double_set::const_iterator, bool> p;
    p = doubleSet.insert(9.5); // insert函数返回值是一个pair对象
    if( p.second )           // 其first是被插入元素的迭代器
        cout << "2) " << * (p.first) << " inserted" << endl;
    else                     // second代表是否成功插入了
        cout << "2) " << * (p.first) << " not inserted" << endl;
    return 0;
}
```

- 1) 2.1 3.7 4.2 9.5
- 2) 9.5 not inserted

- 关联容器：set和multiset
- 关联容器：map和multimap
- 容器适配器
 - stack / queue / priority_queue
- STL中的算法

■ 定义

```
template<class Key, class T, class Pred = less<Key>, class A = allocator<T>>
class multimap {
    ...
    typedef pair<const Key, T> value_type;
    ...
};    //Key 代表关键字
```

- multimap中的元素由<关键字， 值>组成，每个元素是一个pair对象
- multimap中允许多个元素的关键字相同
- 元素按照关键字升序排列，缺省情况下用 less<Key> 定义其"小于"关系

multimap的用法示例

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    typedef multimap<int, double, less<int>> mmid;
    mmid mmpairs;
    cout << "1) " << mmpairs.count(15) << endl;
    mmpairs.insert(mmid::value_type(15, 99.3));
    mmpairs.insert(mmid::value_type(15, 2.7));
    cout << "2) " << mmpairs.count(15) << endl;
    mmpairs.insert(mmid::value_type(30, 111.11));
    mmpairs.insert(mmid::value_type(10, 22.22));
    mmpairs.insert(mmid::value_type(25, 33.333));
    mmpairs.insert(mmid::value_type(20, 9.3));
    for(mmid::const_iterator i = mmpairs.begin(); i != mmpairs.end(); i++)
        cout << "(" << i->first << ", " << i->second << ")" << ", ";
    return 0;
}
```

```
1) 0
2) 2
(10, 22.22), (15, 99.3), (15, 2.7), (20, 9.3), (25, 33.333), (30, 111.11),
```

multimap例题

- 一个学生成绩录入和查询系统，接受以下两种输入：

Add name id score

Query score

- name是个字符串，中间没有空格，代表学生姓名，可能重复
 - id是个整数，代表学号，学号不会重复
 - score是个整数，表示分数，可能重复
-
- 两种输入交替出现
 - 第一种输入表示添加一个学生的信息，记录下学生的姓名、id和分数
 - 第二种输入表示要查询，输出已有记录中分数比score低的最高分获得者的姓名、学号和分数。如果有多个学生都满足条件，就输出学号最大的那个学生的信息。如果找不到满足条件的学生，则输出"Nobody"

multimap例题

输入样例：

Add Jack 12 78

Query 78

Query 81

Add Percy 9 81

Add Marry 8 81

Query 82

Add Tom 11 79

Query 80

Query 81

输出样例：

Nobody

Jack 12 78

Percy 9 81

Tom 11 79

Tom 11 79

multimap例题

```
#include <iostream>
#include <map> //使用multimap需要包含此头文件
#include <string>
using namespace std;
class CStudent {
public:
    struct CInfo { //类的内部还可以定义类
        int id;
        string name;
    };
    int score;
    CInfo info; //学生的其他信息
};
typedef multimap<int, CStudent::CInfo> MAP_STD;
```

multimap例题

```
int main() {
    MAP_STD mp;
    CStudent st;
    string cmd;
    while( cin >> cmd ) {
        if( cmd == "Add" ) {
            cin >> st.info.name >> st.info.id >> st.score ;
            mp.insert(MAP_STD::value_type(st.score,st.info));
            // mp.insert(make_pair(st.score,st.info)); 也可以
        }
        else if( cmd == "Query" ){
            ... // 查询相关的代码
        }
    }
    return 0;
}
```

multimap例题

```
else if( cmd == "Query" ){
    int score;
    cin >> score;
    MAP_STD::iterator p = mp.lower_bound(score);
    if( p!= mp.begin() ) {
        --p;
        score = p->first; //比要查询分数低的最高分
        MAP_STD::iterator maxp = p;
        int maxId = p->second.id;
        for( ; p != mp.begin() && p->first == score; --p) { //遍历所有成绩和score相等的学生
            if( p->second.id > maxId ) { //找到学号最大的学生
                maxp = p;
                maxId = p->second.id ;
            }
        }
        if( p->first == score) { //如果上面循环是因为 p == mp.begin() 而终止，则p指向的元素还要处理
            if( p->second.id > maxId ) {
                maxp = p;
                maxId = p->second.id ;
            }
        }
        cout << maxp->second.name << " " << maxp->second.id << " " << maxp->first << endl;
    }
}
```

iterator **lower_bound**
(const T & val);

查找一个最大的位置 **it**，使得
[begin(),it)中所有元素的
first 都比 **val** 小。

■ 定义

```
template<class Key, class T, class Pred = less<Key>, class A = allocator<T>>
class map {
    ...
    typedef pair<const Key, T> value_type;
    ...
};    //Key 代表关键字
```

- map中的元素由<关键字, 值>组成, 每个元素是一个pair对象
- map中的元素关键字各不相同
- 元素按照关键字升序排列, 缺省情况下用 less<Key> 定义其"小于"关系

map的[]成员函数

- 可以用**pairs[key]**形式访问map中的元素
 - pairs为map容器名，key为关键字的值
 - 该表达式返回的是对关键值为key的元素的值的引用
 - 如果没有关键字为key的元素，则会往pairs里插入一个关键字为key的元素，并返回其值的引用
- 例如：

```
map<int, double> pairs;
```

- 则 pairs[50] = 5; 会修改pairs中关键字为50的元素，使其值变成5
- 若不存在关键字等于50的元素，则插入此元素，并使其值变为5

map的用法示例

《程序设计实习》

```
#include <iostream>
#include <map>
using namespace std;

ostream & operator <<(ostream & o, const pair<int, double> & p){
    o << "(" << p.first << ", " << p.second << ")";
    return o;
}

int main() {
    typedef map<int, double, less<int>> mmid;
    mmid pairs;
    cout << "1) " << pairs.count(15) << endl;
    pairs.insert(mmid::value_type(15, 2.7));
    pairs.insert(make_pair(15, 99.3)); //make_pair生成一pair对象
    cout << "2) " << pairs.count(15) << endl;
```

1) 0

2) 1

map的用法示例

```
pairs.insert(mmid::value_type(20, 9.3));
mmid::iterator i;
cout << "3) ";
for( i = pairs.begin(); i != pairs.end(); i ++ )
    cout << * i << ", ";
cout << endl;
cout << "4) ";
int n = pairs[40];
//如果没有关键字为40的元素，则插入一个
for( i = pairs.begin(); i != pairs.end(); i ++ )
    cout << * i << ", ";
cout << endl;
cout << "5) ";
pairs[15] = 6.28;
//把关键字为15的元素值改成6.28
for( i = pairs.begin(); i != pairs.end(); i ++ )
    cout << * i << ", ";
return 0;
}
```

3) (15, 2.7), (20, 9.3),
4) (15, 2.7), (20, 9.3), (40, 0),
5) (15, 6.28), (20, 9.3), (40, 0),

- 关联容器：set和multiset
- 关联容器：map和multimap
- 容器适配器
 - stack / queue / priority_queue
- STL中的算法

■ 可以用某种顺序容器来实现

(让已有的顺序容器以栈/队列的方式工作)

1) **stack**: 头文件 `<stack>`

栈, 后进先出

2) **queue**: 头文件 `<queue>`

队列, 先进先出

3) **priority_queue**: 头文件 `<queue>`

优先级队列, 最高优先级元素总是第一个出列

容器适配器： stack

《程序设计实习》

- 可用vector, list, deque来实现
 - 缺省情况下，用deque实现
 - 用vector和deque实现，比用list实现性能好

```
template<class T, class Cont = deque<T>>
class stack {
    ...
};
```

- stack是**后进先出**的数据结构，只能插入/删除/访问栈顶的元素

■ stack的使用

```
stack<int> stk; // int型栈，用deque实现
stack<string, vector<string>> str_stk; // string型栈，用vector实现
stack<string, vector<string>> str_stk(svec); // string型栈，用vector实现，
// 并且用向量svec初始化
```

■ stack上可以进行以下操作：

- **push**：插入元素
- **pop**：弹出元素
- **top**：返回栈顶元素的引用

容器适配器：queue

《程序设计实习》

- 和stack基本类似，可以用list和deque实现，缺省情况下用deque实现

```
template<class T, class Cont = deque<T>>
class queue {
    ...
};
```

- 同样也有push，pop，top函数
- 但是push发生在队尾，pop / top发生在队头，先进先出
- 有back成员函数可以返回队尾元素的引用

容器适配器：priority_queue

《程序设计实习》

- 和queue类似，可以用vector和deque实现，缺省情况下用vector实现

```
template <class T, class Container = vector<T>, class Compare = less<T>>
class priority_queue;
```

- priority_queue 通常用堆排序技术实现，保证最大的元素总是在最前面，默认的元素比较器是less<T>
 - 执行pop操作时，删除的是最大的元素 $O(\log n)$
 - 执行top操作时，返回的是最大元素的引用 $O(1)$
 - 执行push操作时，添加一个元素 $O(\log n)$

priority_queue的用法示例

《程序设计实习》

```
#include <queue>
#include <iostream>
using namespace std;
int main() {
    priority_queue<double> priorities;
    priorities.push(3.2);
    priorities.push(9.8);
    priorities.push(5.4);
    while( !priorities.empty() ) {
        //输出最大元素的引用
        cout << priorities.top() << " ";
        priorities.pop();    //删除最大元素
    }
    return 0;
}
```

stack, queue, priority_queue都有
empty(): 用于判断适配器是否为空
size(): 返回适配器中元素个数

9.8 5.4 3.2

- 关联容器：set和multiset
- 关联容器：map和multimap
- 容器适配器
 - stack / queue / priority_queue
- STL中的算法

■ STL中的算法大致可以分为以下七类：

- 不变序列算法
- 变值算法
- 删除算法
- 变序算法
- 排序算法
- 有序区间算法
- 数值算法

- 大多重载的算法都是有**两个版本**的
 - 用 "**==**" 判断元素是否相等，或用 "**<**" 来比较大小
 - 多出一个类型参数 "Pred" 和函数形参 "Pred op" :
通过**表达式** "**op(x, y)**" 的**返回值**: true / false
→ 判断 x 是否 "等于" y, 或者 x 是否 "小于" y
- 如下面的有两个版本的**min_element**:

```
iterator min_element(iterator first, iterator last);  
iterator min_element(iterator first, iterator last, Pred op);
```

- 该类算法不会修改算法所作用的容器或对象
- 适用于所有容器
- 时间复杂度都是 $O(n)$

算法名称	功 能
min	求两个对象中较小的（可自定义比较器）
max	求两个对象中较大的（可自定义比较器）
min_element	求区间中的最小值（可自定义比较器）
max_element	求区间中的最大值（可自定义比较器）
for_each	对区间中的每个元素都做某种操作
count	计算区间中等于某值的元素个数
count_if	计算区间中符合某种条件的元素个数

算法名称	功 能
find	在区间中查找等于某值的元素
find_if	在区间中查找符合某条件的元素
find_end	在区间中查找另一个区间最后一次出现的位置（可自定义比较器）
find_first_of	在区间中查找第一个出现在另一个区间中的元素（可自定义比较器）
adjacent_find	在区间中寻找第一次出现连续两个相等元素的位置（可自定义比较器）
search	在区间中查找另一个区间第一次出现的位置（可自定义比较器）
search_n	在区间中查找第一次出现等于某值的连续n个元素（可自定义比较器）
equal	判断两区间是否相等（可自定义比较器）
mismatch	逐个比较两个区间的元素，返回第一次发生不相等的两个元素的位置（可自定义比较器）
lexicographical_compare	按字典序比较两个区间的大小（可自定义比较器）

for_each

```
template<class InIt, class Fun>  
Fun for_each(InIt first, InIt last, Fun f);
```

- 对[first, last)中的每个元素e，执行f(e)，要求f(e)不能改变e

count

```
template<class InIt, class T>  
size_t count(InIt first, InIt last, const T& val);
```

- 计算[first, last) 中等于 val的元素个数

count_if

```
template<class InIt, class Pred>  
size_t count_if(InIt first, InIt last, Pred pr);
```

- 计算[first, last) 中符合 pr(e) == true 的元素e的个数

min_element

```
template<class FwdIt>  
FwdIt min_element(FwdIt first, FwdIt last);
```

- 返回[first, last)中最小元素的迭代器，以 "<" 作比较器
- 最小指没有元素比它小，而不是它比别的不同元素都小
- 因为即便 $a \neq b$ ， $a < b$ 和 $b < a$ 有可能都不成立

max_element

```
template<class FwdIt>  
FwdIt max_element(FwdIt first, FwdIt last);
```

- 返回[first, last)中最大元素（不小于任何其他元素）的迭代器
- 以 "<" 作比较器

min_element和max_element示例

《程序设计实习》

```
#include <iostream>
#include <algorithm>
using namespace std;
class A {
public:
    int n;
    A(int i):n(i) { }
};
bool operator<( const A & a1, const A & a2) {
    cout << "< called,a1=" << a1.n << " a2=" << a2.n << endl;
    if(a1.n == 3 && a2.n == 7)
        return true;
    return false;
}
int main() {
    A aa[] = {3,5,7,2,1};
    cout << min_element(aa,aa+5)->n << endl;
    cout << max_element(aa,aa+5)->n << endl;
    return 0;
}
```

```
< called,a1=5 a2=3
< called,a1=7 a2=3
< called,a1=2 a2=3
< called,a1=1 a2=3
3
< called,a1=3 a2=5
< called,a1=3 a2=7
< called,a1=7 a2=2
< called,a1=7 a2=1
7
```

find

```
template<class InIt, class T>  
InIt find(InIt first, InIt last, const T& val);
```

- 返回区间 $[first, last)$ 中的迭代器 i , 使得 $*i == val$

find_if

```
template<class InIt, class Pred>  
InIt find_if(InIt first, InIt last, Pred pr);
```

- 返回区间 $[first, last)$ 中的迭代器 i , 使得 $pr(*i) == true$

- 此类算法会修改源区间或目标区间元素的值
- 值被修改的那个区间，不可以是属于关联容器的

算法名称	功 能
for_each	对区间中的每个元素都做某种操作
copy	复制一个区间到别处
copy_backward	复制一个区间到别处，但目标区间是从后往前被修改的
transform	将一个区间的元素变形后拷贝到另一个区间

算法名称	功 能
swap_ranges	交换两个区间内容
fill	用某个值填充区间
fill_n	用某个值替换区间中的n个元素
generate	用某个操作的结果填充区间
generate_n	用某个操作的结果替换区间中的n个元素
replace	将区间中的某个值替换为另一个值
replace_if	将区间中符合某种条件的值替换成另一个值
replace_copy	将一个区间拷贝到另一个区间，拷贝时某个值要换成新值拷过去
replace_copy_if	将一个区间拷贝到另一个区间，拷贝时符合某条件的值要换成新值拷过去

transform

```
template<class InIt, class OutIt, class Unop>  
OutIt transform(InIt first, InIt last, OutIt x, Unop uop);
```

- 对[first, last)中的每个迭代器I,
 - 执行 uop(* I); 并将结果依次放入从 x 开始的地方
 - 要求 uop(* I) 不得改变 * I 的值
- 本模板返回值是个迭代器, 即 x + (last-first)
 - x可以和first相等

样例程序

```
#include <vector>
#include <iostream>
#include <numeric>
#include <list>
#include <algorithm>
#include <iterator>
using namespace std;
class CLessThan9 {
public:
    bool operator()( int n) { return n < 9; }
};
void outputSquare(int value ) { cout << value * value << " "; }
int calculateCube(int value) { return value * value * value; }

int main() {
    const int SIZE = 10;
    int a1[] = { 1,2,3,4,5,6,7,8,9,10 };
    int a2[] = { 100,2,8,1,50,3,8,9,10,2 };
    vector<int> v(a1, a1+SIZE);
    ostream_iterator<int> output(cout, " ");
    random_shuffle(v.begin(), v.end());
```

样例程序

```
cout << endl << "1) ";
copy( v.begin(), v.end(), output);
copy( a2, a2+SIZE, v.begin());
cout << endl << "2) ";
cout << count(v.begin(), v.end(), 8);
cout << endl << "3) ";
cout << count_if(v.begin(), v.end(), CLessThan9());
cout << endl << "4) ";
cout << * (min_element(v.begin(), v.end()));
cout << endl << "5) ";
cout << * (max_element(v.begin(), v.end()));
cout << endl << "6) ";
cout << accumulate(v.begin(), v.end(), 0); //求和
cout << endl << "7) ";
for_each(v.begin(), v.end(), outputSquare);
vector<int> cubes(SIZE);
transform(a1, a1+SIZE, cubes.begin(), calculateCube);
cout << endl << "8) ";
copy(cubes.begin(), cubes.end(), output);
return 0;
```

```
}
```

// a1 1,2,3,4,5,6,7,8,9,10

// a2 100,2,8,1,50,3,8,9,10,2

1) 9 2 10 3 1 6 8 4 5 7 // 随机的

2) 2

3) 6

4) 1

5) 100

6) 193

7) 10000 4 64 1 2500 9 64 81 100 4

8) 1 8 27 64 125 216 343 512 729 1000

- 删除一个容器里的某些元素
- 这里的"删除"不会使容器里的元素减少
- 工作过程：
 - 将所有应该被删除的元素看做空位子
 - 用留下的元素从后往前移，依次去填空位子
 - 元素往前移后，它原来的位置也就算是空位子
 - 也应由后面的留下的元素来填上
 - 最后，没有被填上的空位子，维持其原来的值不变
- 删除算法不应作用于关联容器

删除算法

算法名称	功 能
remove	删除区间中等于某个值的元素
remove_if	删除区间中满足某种条件的元素
remove_copy	拷贝区间到另一个区间，等于某个值的元素不拷贝
remove_copy_if	拷贝区间到另一个区间，符合某种条件的元素不拷贝
unique	删除区间中连续相等的元素，只留下一个(可自定义比较器)
unique_copy	拷贝区间到另一个区间，连续相等的元素，只拷贝第一个到目标区间(可自定义比较器)

unique

```
template<class FwdIt>
FwdIt unique(FwdIt first, FwdIt last);
```

- 用 == 比较是否等

```
template<class FwdIt, class Pred>
FwdIt unique(FwdIt first, FwdIt last, Pred pr);
```

- 用 pr 比较是否等
- 对[first, last) 这个序列中连续相等的元素，只留下第一个
- 返回值是迭代器，指向元素删除后的区间的最后一个元素的后面

remove 样例程序

```
#include <vector>
#include <iostream>
#include <algorithm>
#include <iterator>
using namespace std;

int main(){
    int a[5] = { 1,2,3,2,5 };
    int b[6] = { 1,2,3,2,5,6 };
    ostream_iterator<int> oit(cout, ",");
    int * p = remove(a, a+5, 2);
    cout << "1) "; copy(a, a+5, oit); cout << endl;
    cout << "2) " << p - a << endl;
    vector<int> v(b, b+6);
    remove(v.begin(), v.end(), 2);
    cout << "3) "; copy(v.begin(), v.end(), oit); cout << endl;
    cout << "4) "; cout << v.size() << endl;
    //v中的元素没有减少
    return 0;
}
```

- 1) 1,3,5,2,5,
- 2) 3
- 3) 1,3,5,6,5,6,
- 4) 6

- 变序算法改变容器中元素的顺序，但是不改变元素的值
- 变序算法不适用于关联容器，算法复杂度都是 $O(n)$ 的

算法名称	功 能
reverse	颠倒区间的前后次序
reverse_copy	把一个区间颠倒后的结果拷贝到另一个区间，源区间不变
rotate	将区间进行循环左移
rotate_copy	将区间以首尾相接的形式进行旋转后的结果拷贝到另一个区间，源区间不变
next_permutation	将区间改为下一个排列（可自定义比较器）
prev_permutation	将区间改为上一个排列（可自定义比较器）
random_shuffle	随机打乱区间内元素的顺序
partition	把区间内满足某个条件的元素移到前面，不满足该条件的移到后面

变序算法

算法名称	功 能
<code>stable_partition</code>	把区间内满足某个条件的元素移到前面 不满足该条件的移到后面 而对这两部分元素，分别保持它们原来的先后次序不变

random_shuffle

```
template<class RanIt>  
void random_shuffle(RanIt first, RanIt last);
```

- 随机打乱[first, last)中的元素, 适用于能随机访问的容器

reverse

```
template<class BidIt>  
void reverse(BidIt first, BidIt last);
```

- 颠倒区间[first, last)顺序

next_permutation

```
template<class Init>  
bool next_permutation (Init first, Init last);
```

- 求下一个排列

next_permutation 样例程序1

《程序设计实习》

```
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;
int main(){
    string str = "231";
    char szStr[] = "324";
    while (next_permutation(str.begin(), str.end())){
        cout << str << endl;
    }
    cout << "*****" << endl;
    while (next_permutation(szStr, szStr + 3)){
        cout << szStr << endl;
    }
    sort(str.begin(), str.end());
    cout << "*****" << endl;
    while (next_permutation(str.begin(), str.end())){
        cout << str << endl;
    }
    return 0;
}
```

312

321

342

423

432

132

213

231

312

321

next_permutation 样例程序2

《程序设计实习》

```
#include <iostream>
#include <algorithm>
#include <string>
#include <list>
#include <iterator>
using namespace std;
int main(){
    int a[] = { 8,7,10 };
    list<int> ls(a, a+3);
    while( next_permutation( ls.begin(), ls.end() )) {
        list<int>::iterator i;
        for( i = ls.begin(); i != ls.end(); ++i)
            cout << * i << " ";
        cout << endl;
    }
}
```

```
8 10 7
10 7 8
10 8 7
```


- 比前面的变序算法复杂度更高，一般是 $O(n\log(n))$
- 排序算法需要随机访问迭代器的支持，不适用于关联容器和list

算法名称	功 能
sort	将区间从小到大排序（可自定义比较器）
stable_sort	将区间从小到大排序 并保持相等元素间的相对次序（可自定义比较器）
partial_sort	对区间部分排序，直到前n个元素就位（可自定义比较器）
partial_sort_copy	将区间前n个元素的排序结果拷贝到别处 源区间不变（可自定义比较器）
nth_element	对区间部分排序，使得第n小的元素（n从0开始算）就位， 而且比它小的都在它前面，比它大的都在它后面（可自定义比较器）

- 比前面的变序算法复杂度更高，一般是 $O(n\log(n))$
- 排序算法需要随机访问迭代器的支持，不适用于关联容器和list

算法名称	功 能
make_heap	使区间成为一个 "堆" (可自定义比较器)
push_heap	将元素加入一个是 "堆" 区间 (可自定义比较器)
pop_heap	从 "堆" 区间删除堆顶元素 (可自定义比较器)
sort_heap	将一个 "堆" 区间进行排序， 排序结束后，该区间就是普通的有序区间， 不再是 "堆" 了 (可自定义比较器)

sort 快速排序

```
template<class RanIt>
void sort(RanIt first, RanIt last);
```

- 按升序排序
- 判断x是否应比y靠前，就看 $x < y$ 是否为true

```
template<class RanIt, class Pred>
void sort(RanIt first, RanIt last, Pred pr);
```

- 按升序排序
- 判断x是否应比y靠前，就看 $pr(x, y)$ 是否为true

sort 样例程序

《程序设计实习》

```
#include <iostream>
#include <algorithm>
using namespace std;
class MyLess {
public:
    bool operator()( int n1,int n2) {
        return (n1 % 10) < ( n2 % 10);
    }
};
int main() {
    int a[] = { 14,2,9,111,78 };
    sort(a, a + 5, MyLess());
    int i;
    for(i = 0; i < 5; i ++){
        cout << a[i] << " ";
    }
    cout << endl;
    sort(a, a+5, greater<int>());
    for(i = 0; i < 5; i ++){
        cout << a[i] << " ";
    }
    return 0;
}
```

111 2 14 78 9

111 78 14 9 2

排序算法

- **sort** 实际上是快速排序，时间复杂度 $O(n\log(n))$
 - 平均性能最优
 - 但是最坏的情况下，性能可能非常差
- 如果要保证"最坏情况下"的性能，那么可以使用 **stable_sort**
 - **stable_sort** 是归并排序，特点是能保持相等元素之间的先后次序
 - 在有足够存储空间的情况下，复杂度为 $n\log(n)$ ，否则为 $n\log^2(n)$
 - **stable_sort** 用法和 **sort** 相同
- 排序算法要求**随机存取迭代器**的支持，所以**list**不能使用排序算法，要使用**list::sort**

堆排序

- **堆**：一种二叉树，最大元素总是在堆顶上，二叉树中任何节点的子节点总是小于或等于父节点的值
- **什么是堆**
 - n 个记录的序列，其所对应的关键字的序列为 $\{k_0, k_1, k_2, \dots, k_{n-1}\}$ ，若有如下关系成立时，则称该记录序列构成一个堆。
 $k_i \geq k_{2i+1}$ 且 $k_i \geq k_{2i+2}$ ，其中 $i = 0, 1, \dots$
 - 例如：96 83 27 38 11 9
- 堆排序的各种算法，如`make_heap`，需要随机访问迭代器的支持

make_heap

```
template<class RanIt>
void make_heap(RanIt first, RanIt last);
```

- 将区间 [first,last) 做成一个堆
- 用 < 作比较器

```
template<class RanIt, class Pred>
void make_heap(RanIt first, RanIt last, Pred pr);
```

- 将区间 [first,last) 做成一个堆
- 用 pr 作比较器

push_heap

```
template<class RanIt>
void push_heap(RanIt first, RanIt last);

template<class RanIt, class Pred>
void push_heap(RanIt first, RanIt last, Pred pr);
```

- 在 $[first, last-1)$ 已经是堆的情况下，将 $[first, last)$ 变成堆，时间复杂度 $O(\log(n))$
- 往已经是堆的容器中添加元素，可以在每次 `push_back` 一个元素后，再调用 `push_heap` 算法

pop_heap

```
template<class RanIt>
void pop_heap(RanIt first, RanIt last);

template<class RanIt, class Pred>
void pop_heap(RanIt first, RanIt last, Pred pr);
```

- 取出堆中最大的元素，复杂度 $O(\log(n))$
 - 将堆中的最大元素，即 `* first`，移到 `last - 1` 位置，
 - 原 `* (last - 1)` 被移到前面某个位置，并且移动后 `[first, last - 1)` 仍然是个堆
 - 要求原 `[first, last)` 就是个堆

- 要求所操作的区间是已经从小到大排好序的
- 需要随机访问迭代器的支持，不适用于关联容器和list

算法名称	功 能
<code>binary_search</code>	判断区间中是否包含某个元素
<code>includes</code>	判断是否一个区间中的每个元素，都在另一个区间中
<code>lower_bound</code>	查找最后一个不小于某值的元素的位置
<code>upper_bound</code>	查找第一个大于某值的元素的位置
<code>equal_range</code>	同时获取 <code>lower_bound</code> 和 <code>upper_bound</code>
<code>merge</code>	合并两个有序区间到第三个区间

- 要求所操作的区间是已经从小到大排好序的
- 需要随机访问迭代器的支持，不适用于关联容器和list

算法名称	功 能
set_union	将两个有序区间的并拷贝到第三个区间
set_intersection	将两个有序区间的交拷贝到第三个区间
set_difference	将两个有序区间的差拷贝到第三个区间
set_symmetric_difference	将两个有序区间的对称差拷贝到第三个区间
inplace_merge	将两个连续的有序区间原地合并为一个有序区间

binary_search

```
template<class FwdIt, class T>
bool binary_search(FwdIt first, FwdIt last, const T& val);

template<class FwdIt, class T, class Pred>
bool binary_search(FwdIt first, FwdIt last, const T& val, Pred pr);
```

- 折半查找
- 要求容器已经**有序**且支持随机访问迭代器，返回是否找到
 - 不带pr的版本，比较两个元素x, y 大小时，看 $x < y$
 - 带pr的版本，比较两个元素x, y 大小时，若 $pr(x, y)$ 为true，则认为x小于y

binary_search 样例程序

《程序设计实习》

```
bool Greater10(int n) {  
    return n > 10;  
}  
  
int main() {  
    const int SIZE = 10;  
    int a1[] = { 2,8,1,50,3,100,8,9,10,2 };  
    vector<int> v(a1, a1+SIZE);  
    vector<int>::iterator location;  
    location = find(v.begin(), v.end(), 10);  
    if(location != v.end()) {  
        cout << endl << "1) " << location - v.begin();  
    }  
    location = find_if(v.begin(), v.end(), Greater10);  
    if(location != v.end())  
        cout << endl << "2) " << location - v.begin();  
    sort(v.begin(), v.end());  
    if(binary_search(v.begin(), v.end(), 9)) {  
        cout << endl << "3) " << "9 found";  
    }  
    return 0;  
}
```

1) 8
2) 3
3) 9 found

lower_bound

```
template<class FwdIt, class T>  
FwdIt lower_bound(FwdIt first, FwdIt last, const T& val);
```

- 要求[first, last) 是有序的
- 查找[first, last) 中的，最大的位置 FwdIt，使得[first, FwdIt) 中所有的元素都比 val 小

upper_bound

```
template<class FwdIt, class T>  
FwdIt upper_bound(FwdIt first, FwdIt last, const T& val);
```

- 要求[first, last) 是有序的
- 查找[first, last) 中的，最小的位置 FwdIt，使得[first, last) 中所有的元素都比 val 大

equal_range

```
template<class FwdIt, class T>  
pair<FwdIt, FwdIt> equal_range(FwdIt first, FwdIt last, const T& val);
```

- 要求[first, last) 是有序的
- 返回值是一个pair, 假设为 p, 则:
 - [first, p.first) 中的元素都比 val 小
 - [p.second, last) 中的所有元素都比 val 大
 - p.first 就是lower_bound的结果
 - p.last 就是upper_bound的结果

merge

```
template<class InIt1, class InIt2, class OutIt>
OutIt merge(InIt1 first1, InIt1 last1, InIt2 first2, InIt2 last2, OutIt x);

template<class InIt1, class InIt2, class OutIt, class Pred>
OutIt merge(InIt1 first1, InIt1 last1, InIt2 first2, InIt2 last2, OutIt x, Pred pr);
```

- 把[first1, last1), [first2, last2) 两个升序序列合并，形成第3个升序序列，第3个升序序列以 x 开头
 - 第一个用 < 作比较器
 - 第二个用 pr 作比较器

includes

```
template<class InIt1, class InIt2>
bool includes(InIt1 first1, InIt1 last1, InIt2 first2, InIt2 last2);

template<class InIt1, class InIt2, class Pred>
bool includes(InIt1 first1, InIt1 last1, InIt2 first2, InIt2 last2, Pred pr);
```

- 要求2个容器都是有序的
- 判断 `[first2, last2)` 中的每个元素，是否都在 `[first1, last1)` 中
 - 第一个用 `<` 作比较器
 - 第二个用 `pr` 作比较器，`pr(x, y) == true`说明 `x, y`相等

set_difference

```
template<class InIt1, class InIt2, class OutIt>
OutIt set_difference(InIt1 first1, InIt1 last1,
                    InIt2 first2, InIt2 last2, OutIt x);

template<class InIt1, class InIt2, class OutIt, class Pred>
OutIt set_difference(InIt1 first1, InIt1 last1,
                    InIt2 first2, InIt2 last2, OutIt x, Pred pr);
```

- 求出[first1, last1)中，不在[first2, last2)中的元素，放到从x开始的地方
- 如果 [first1, last1) 里有多多个相等元素不在[first2, last2)中，则这多个元素也都会被放入x代表的目标区间里

set_intersection

```
template<class InIt1, class InIt2, class OutIt>
OutIt set_intersection(InIt1 first1, InIt1 last1,
                      InIt2 first2, InIt2 last2, OutIt x);

template<class InIt1, class InIt2, class OutIt, class Pred>
OutIt set_intersection(InIt1 first1, InIt1 last1,
                      InIt2 first2, InIt2 last2, OutIt x, Pred pr);
```

- 求出[first1, last1)和[first2, last2)中共有的元素，放到从x开始的地方
- 若某个元素e 在[first1, last1)里出现n1次，在[first2, last2)里出现n2次，则该元素在目标区间里出现min(n1, n2)次

set_symmetric_difference

```
template<class InIt1, class InIt2, class OutIt>
OutIt set_symmetric_difference(InIt1 first1, InIt1 last1,
                              InIt2 first2, InIt2 last2, OutIt x);

template<class InIt1, class InIt2, class OutIt, class Pred>
OutIt set_symmetric_difference(InIt1 first1, InIt1 last1,
                              InIt2 first2, InIt2 last2, OutIt x, Pred pr);
```

- 把两个区间里相互不在另一区间里的元素放入x开始的地方

set_union

```
template<class InIt1, class InIt2, class OutIt>
OutIt set_union(InIt1 first1, InIt1 last1,
               InIt2 first2, InIt2 last2, OutIt x);

template<class InIt1, class InIt2, class OutIt, class Pred>
OutIt set_union(InIt1 first1, InIt1 last1,
               InIt2 first2, InIt2 last2, OutIt x, Pred pr);
```

- 求两个区间的并，放到以x开始的位置
- 若某个元素 e 在 $[first1, last1)$ 里出现 $n1$ 次，在 $[first2, last2)$ 里出现 $n2$ 次，则该元素在目标区间里出现 $\max(n1, n2)$ 次

bitset容器类

```
template<size_t N>
class bitset
{
    ...
};
```

- 实际使用的时候，N是个整型常数
- 例如
 - `bitset<40> bst;`
 - `bst`是一个由40位组成的对象
 - 用`bitset`的函数可以方便地访问任何一位

bitset容器类

■ `bitset`的成员函数：

```
bitset<N>& operator&=(const bitset<N>& rhs);  
bitset<N>& operator|=(const bitset<N>& rhs);  
bitset<N>& operator^=(const bitset<N>& rhs);  
bitset<N>& operator<<=(size_t num);  
bitset<N>& operator>>=(size_t num);  
bitset<N>& set(); //全部设成1  
bitset<N>& set(size_t pos, bool val = true); //设置某位  
bitset<N>& reset(); //全部设成0  
bitset<N>& reset(size_t pos); //某位设成0  
bitset<N>& flip(); //全部翻转  
bitset<N>& flip(size_t pos); //翻转某位
```


bitset容器类

■ **bitset**的成员函数：

```
reference operator[](size_t pos);    //返回对某位的引用
bool operator[](size_t pos) const;   //判断某位是否为1
reference at(size_t pos);
bool at(size_t pos) const;
unsigned long to_ulong() const;      //转换成整数
string to_string() const;            //转换成字符串
size_t count() const;                //计算1的个数
size_t size() const;
bool operator==(const bitset<N>& rhs) const;
bool operator!=(const bitset<N>& rhs) const;
```

bitset容器类

■ bitset的成员函数：

```
bool test(size_t pos) const;           //测试某位是否为 1
bool any() const;                       //是否有某位为1
bool none() const;                     //是否全部为0
bitset<N> operator<<(size_t pos) const;
bitset<N> operator>>(size_t pos) const;
bitset<N> operator~();
static const size_t bitset_size = N;
// 注意：第0位在最右边
```